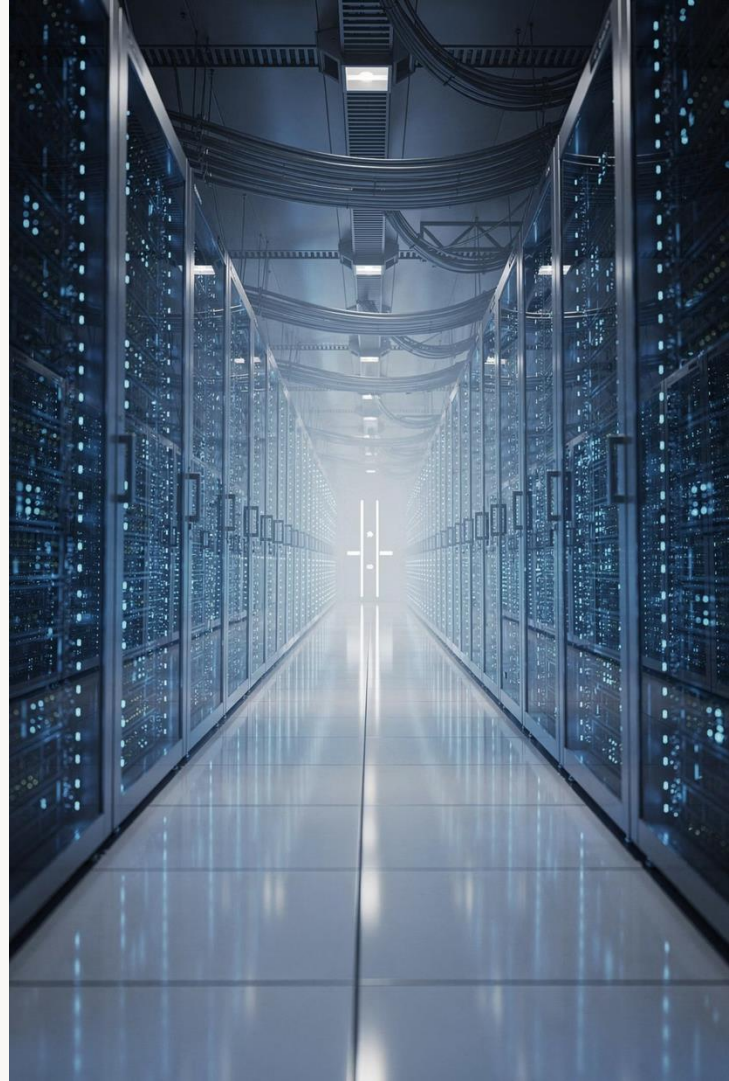


Seguridad Aplicada a Entornos Java & .NET

Sesión 1 — Fundamentos de identidad y control de acceso

Curso avanzado de seguridad backend · 16 horas · Dirigido a desarrolladores/as senior, tech leads y arquitectos/as



Sobre este curso

Objetivo general

Desarrollar competencias sólidas en seguridad backend, abordando autenticación, autorización, protección de APIs y mitigación de vulnerabilidades comunes en aplicaciones modernas sobre Java y .NET.

Perfil del destinatario

- Desarrolladores/as con experiencia en backend
- Perfiles senior y tech leads
- Arquitectos/as de sistemas distribuidos

Duración total

16 horas distribuidas en sesiones prácticas y teóricas.

Estructura del curso completo

01

Sesión 1 · 3h 20min

Fundamentos de identidad y control de acceso: autenticación, autorización, SSO, OAuth2, OIDC, JWT.

03

Sesión 3

Seguridad en APIs: rate limiting, CORS, SSL/TLS, headers, gateways, WAF y gestión de secretos.

02

Sesión 2

Spring Security y ASP.NET Identity. Configuración avanzada, flujos OAuth2 y gestión de tokens.

04

Sesión 4

Vulnerabilidades comunes, auditoría, logging de acceso y simulación de ataques.

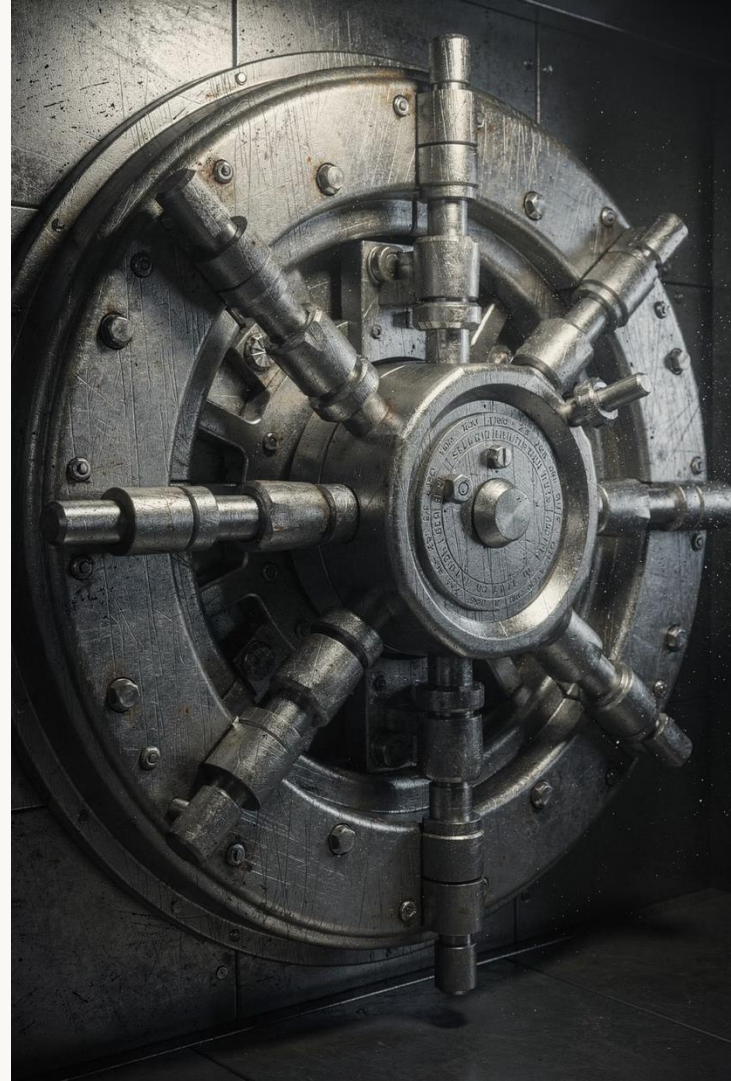
Sesión 1 — Agenda

Duración total: 3 horas 20 minutos



Bloque 1

Autenticación y autorización en aplicaciones modernas



¿Son lo mismo autenticación y autorización?

Uno de los errores conceptuales más frecuentes en el diseño de sistemas seguros es tratar estos dos conceptos como sinónimos. Son procesos distintos que ocurren en momentos distintos y con objetivos distintos.

Autenticación (AuthN)

¿Quién eres? Proceso de verificar la identidad de un usuario, servicio o sistema. Se produce siempre antes que la autorización.

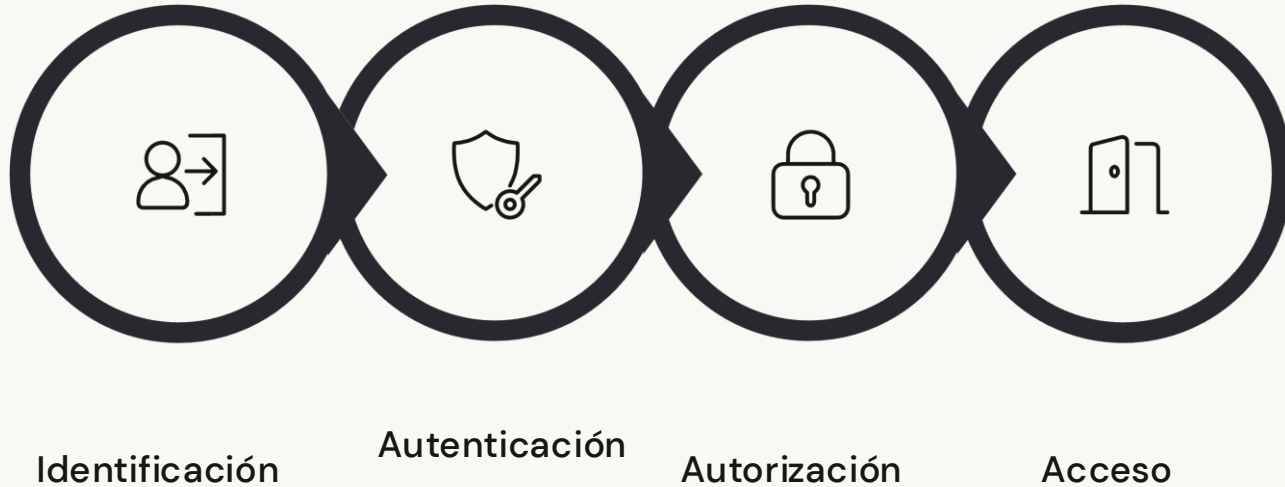
- Usuario/contraseña, certificado, biometría
- MFA, tokens de hardware
- Resultado: identidad verificada (principal)

Autorización (AuthZ)

¿Qué puedes hacer? Proceso de determinar qué operaciones o recursos puede utilizar una identidad ya verificada.

- Roles, permisos, scopes, claims
- Políticas de acceso granular (RBAC, ABAC)
- Resultado: acceso concedido o denegado

Flujo de autenticación y autorización



La autenticación siempre precede a la autorización. Un usuario puede estar correctamente autenticado y aun así no tener autorización para acceder a un recurso concreto. Diseñar estos dos flujos de forma independiente es clave para sistemas seguros y mantenibles.

Mecanismos comunes de autenticación



Credenciales

Usuario y contraseña. El mecanismo más extendido, aunque también el más vulnerable si no se aplican políticas robustas (hashing con bcrypt/Argon2, políticas de complejidad).



Certificados x.509

Muy utilizados en comunicaciones máquina a máquina (mTLS). El cliente y el servidor se autentican mutuamente mediante certificados digitales firmados por una CA.



MFA / 2FA

Autenticación multifactor: combina algo que sabes, algo que tienes y algo que eres. Reduce drásticamente el riesgo de compromiso de cuentas.



Biometría

Huella dactilar, reconocimiento facial o de voz. Cada vez más presente en aplicaciones móviles y en flujos FIDO2/WebAuthn.

Modelos de autorización

RBAC — Role-Based Access Control

Los permisos se asignan a roles, y los usuarios heredan los permisos de sus roles. Sencillo de implementar y auditar. Ideal para sistemas con perfiles bien definidos.

ABAC — Attribute-Based Access Control

Los permisos se evalúan en función de atributos del usuario, del recurso y del contexto (hora, IP, departamento). Más flexible y expresivo que RBAC, pero más complejo de gestionar.

ReBAC — Relationship-Based Access Control

Los permisos dependen de las relaciones entre entidades (ej. «el propietario del documento puede editarlo»). Usado por Google Zanzibar y sistemas como Google Drive.

Scopes y Claims (OAuth2/OIDC)

Permisos granulares declarados en tokens. El servidor de autorización emite scopes que el resource server valida. Muy adecuado para arquitecturas de microservicios.

Arquitecturas de identidad centralizada

En sistemas distribuidos, la gestión descentralizada de identidades genera inconsistencias, duplicidad de lógica y superficies de ataque adicionales. La tendencia actual es centralizar la identidad en un **Identity Provider (IdP)** dedicado.

Identity Provider (IdP)

Entidad de confianza que gestiona las identidades, emite tokens y verifica credenciales. Ejemplos: Keycloak, Auth0, Azure AD, Okta, AWS Cognito.

Service Provider (SP)

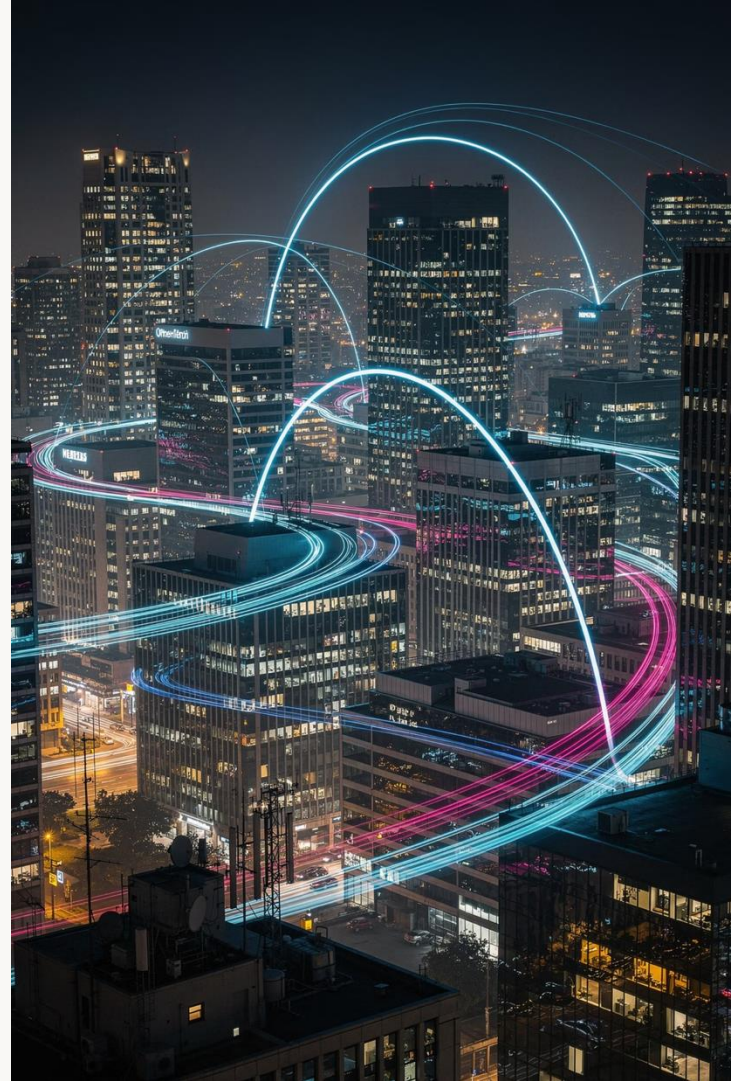
Aplicación o servicio que delega la autenticación al IdP. Confía en los tokens emitidos por el IdP en lugar de gestionar credenciales propias.

Directory Service

Almacén de identidades subyacente: LDAP, Active Directory o una base de datos de usuarios. El IdP consume el directorio para validar identidades.

Identidad centralizada vs. distribuida

Centralizar la identidad en un IdP reduce la superficie de ataque, simplifica el ciclo de vida de las credenciales y permite aplicar políticas de seguridad de forma consistente en todos los servicios.



Beneficios de la identidad centralizada



Política de seguridad unificada

MFA, políticas de contraseña y reglas de acceso se definen una sola vez y se aplican a todos los servicios de forma consistente.



Ciclo de vida simplificado

Alta, baja y modificación de usuarios en un único punto. La revocación de acceso es inmediata y se propaga a todos los servicios.



Auditoría centralizada

Todos los eventos de autenticación quedan registrados en un único sistema, facilitando la detección de anomalías y el cumplimiento normativo (GDPR, SOC2).

Single Sign-On (SSO)

SSO permite que un usuario se autentique una sola vez y acceda a múltiples aplicaciones sin volver a introducir sus credenciales. Es la consecuencia natural de una arquitectura de identidad centralizada.

¿Cómo funciona?

1. El usuario accede a la aplicación A.
2. La aplicación redirige al IdP si no existe sesión.
3. El IdP autentica al usuario y emite un token/aserción.
4. El usuario accede a la aplicación B sin reautenticarse: el IdP ya tiene sesión activa.

Protocolos que lo implementan

- **SAML 2.0** — entornos empresariales, XML
- **OpenID Connect** — moderno, JSON/JWT
- **CAS** — entornos universitarios/legacy
- **WS-Federation** — ecosistema Microsoft

SSO: ventajas e inconvenientes



Ventajas

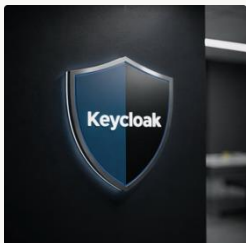
- Mejor experiencia de usuario (UX)
- Reducción de fatiga de contraseñas
- Gestión centralizada de sesiones
- Auditoría y trazabilidad simplificadas
- Facilita la adopción de MFA en toda la organización



Inconvenientes y riesgos

- Single point of failure: si el IdP cae, todo cae
- Compromiso del IdP implica acceso a todas las apps
- Complejidad en la gestión de sesiones distribuidas
- Logout global (SLO) difícil de implementar correctamente

Proveedores de identidad más utilizados



Keycloak

Solución open source de Red Hat. Muy completa: soporta OAuth2, OIDC, SAML, MFA y federación de identidades. Ideal para entornos on-premise.



Azure AD / Entra ID

IdP cloud de Microsoft. Integración nativa con el ecosistema M365 y Azure. Muy extendido en entornos empresariales con stack .NET.



Okta / Auth0

Plataformas IDaaS (Identity as a Service) líderes del mercado. Auth0 está más orientada a desarrolladores; Okta a la gestión empresarial de accesos (IAM).

Bloque 2

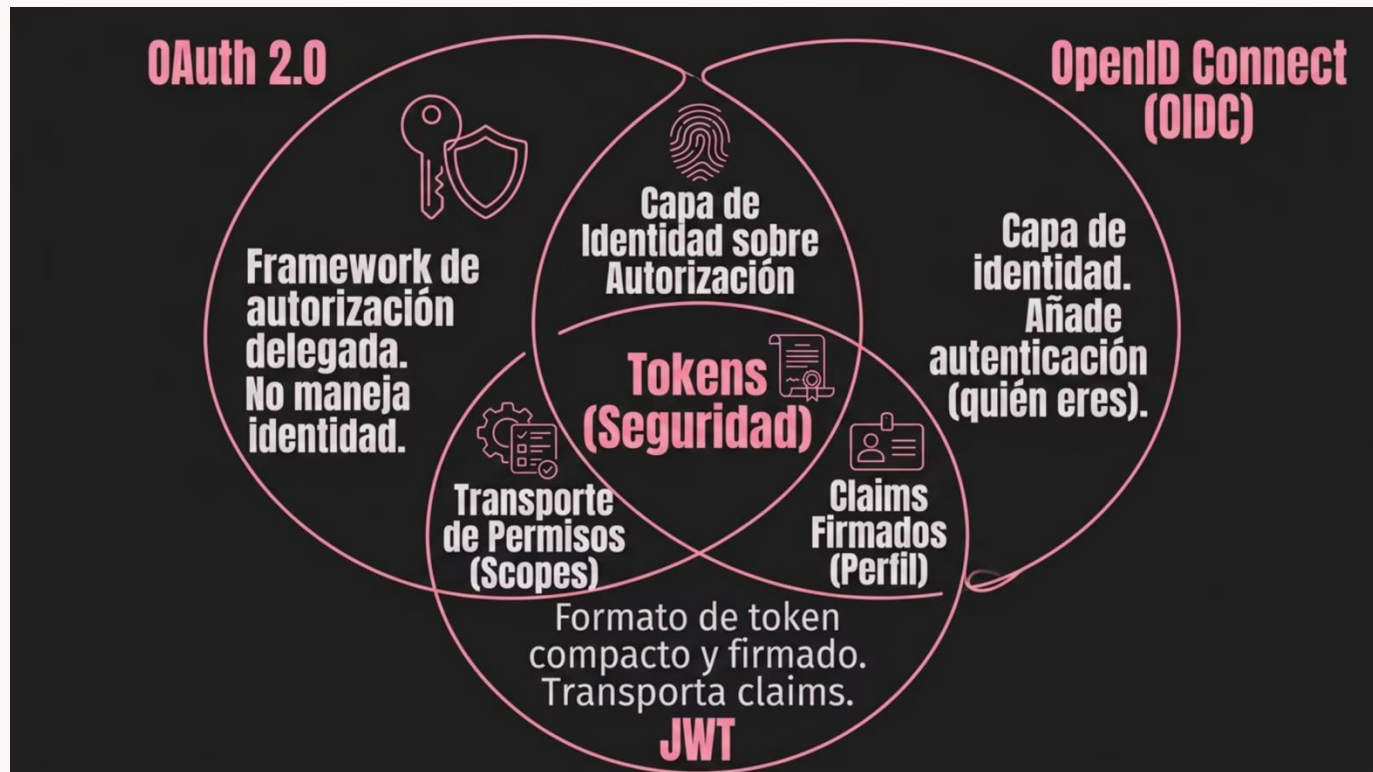
Protocolos y estándares de seguridad

OAuth 2.0 · OpenID Connect · JWT



El ecosistema de estándares de identidad

Los estándares modernos de seguridad forman un ecosistema complementario, no competitivo. Comprender el rol de cada uno es esencial para diseñar arquitecturas correctas.



OAuth 2.0 — ¿Qué es exactamente?

- ❗ OAuth 2.0 es un **framework de autorización delegada**, no un protocolo de autenticación. Su propósito original es permitir que una aplicación acceda a recursos de otra en nombre del usuario, sin que el usuario comparta sus credenciales.

Definido en el **RFC 6749**, OAuth 2.0 introduce el concepto de **acceso delegado**: el usuario (Resource Owner) autoriza a una aplicación (Client) a acceder a sus recursos en un servidor (Resource Server), utilizando tokens emitidos por un Authorization Server.

Ejemplo clásico: una app de terceros que accede a tus contactos de Google sin que le des tu contraseña de Gmail.

Roles en OAuth 2.0

Resource Owner

El usuario (o sistema) propietario del recurso protegido. Es quien otorga o deniega el acceso a sus datos.

Client

La aplicación que solicita acceso a los recursos. Puede ser una SPA, una app móvil, un servicio backend o un CLI.

Authorization Server

Emite tokens de acceso tras verificar la identidad del usuario y obtener su consentimiento. Ej.: Keycloak, Auth0, Azure AD.

Resource Server

Servidor que aloja los recursos protegidos (ej. tu API REST). Valida el access token recibido antes de servir la respuesta.

Tipos de tokens en OAuth 2.0

Access Token

Credencial de corta duración que el cliente presenta al Resource Server para acceder a los recursos. Puede ser opaco (referencia) o estructurado (JWT).

Vida útil recomendada: 5–15 minutos.

Refresh Token

Token de larga duración utilizado para obtener nuevos access tokens sin reautenticar al usuario. Debe almacenarse de forma segura (nunca en localStorage).

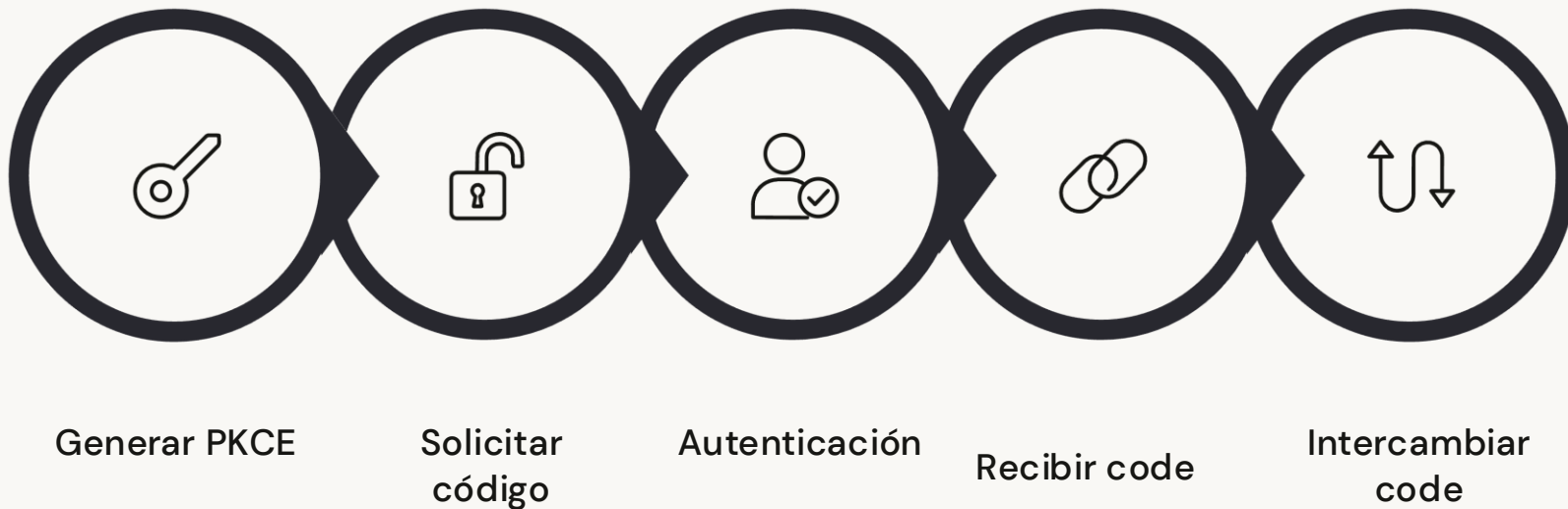
Vida útil: horas, días o semanas según el caso de uso.

Authorization Code

Código de un solo uso intercambiado por tokens. Nunca es el token final; es un paso intermedio en el flujo Authorization Code.

Flujo Authorization Code con PKCE

El flujo más seguro para aplicaciones que interactúan con el usuario. **PKCE** (Proof Key for Code Exchange, RFC 7636) añade protección contra el robo del authorization code, especialmente importante en clientes públicos (SPAs, apps móviles).



Flujo Client Credentials

Diseñado para comunicaciones **máquina a máquina** (M2M) donde no hay usuario involucrado. El client actúa en nombre propio, no en nombre de un usuario.

1

Client solicita token

Envía `client_id` y `client_secret` al token endpoint del Authorization Server.

2

Authorization Server valida

Comprueba las credenciales del cliente y, si son válidas, emite un access token con los scopes solicitados.

3

Client llama al API

Incluye el access token en la cabecera `Authorization: Bearer <token>`. El Resource Server lo valida.

⚠ En este flujo no existe refresh token. Cuando el access token expira, el cliente solicita uno nuevo con sus credenciales.

Comparativa de flujos OAuth 2.0

Flujo	Caso de uso	Tipo de cliente	¿Hay usuario?
Authorization Code + PKCE	Apps web, SPA, móvil	Público o confidencial	Sí
Client Credentials	Microservicios, jobs, daemons	Confidencial	No
Device Code	Smart TVs, CLIs, IoT	Público	Sí (en otro device)
Implicit (deprecated)	Legacy SPAs	Público	Sí
Resource Owner Password (deprecated)	Migraciones legacy	Confidencial	Sí

❌ Los flujos Implicit y Resource Owner Password están **deprecados** en OAuth 2.1. No los utilices en sistemas nuevos.

OpenID Connect (OIDC)

OpenID Connect 1.0 es una capa de identidad construida sobre OAuth 2.0. Mientras OAuth 2.0 responde a «¿qué puede hacer esta app?», OIDC responde a «¿quién es este usuario?»

Qué añade OIDC a OAuth 2.0

- **ID Token:** JWT que contiene claims de identidad del usuario
- **UserInfo Endpoint:** endpoint estándar para obtener claims adicionales
- **Scopes estándar:** openid, profile, email, address, phone
- **Discovery Document:** /.well-known/openid-configuration
- **JWKS Endpoint:** claves públicas para verificar tokens

ID Token vs Access Token

ID Token: para la aplicación cliente. Contiene info del usuario (sub, name, email). *Nunca* debe enviarse a una API.

Access Token: para el Resource Server. Contiene scopes y claims de autorización. La API lo valida.

Claims estándar en OIDC

Claims de identidad básicos

`sub` (subject, identificador único), `name`, `given_name`, `family_name`, `email`, `email_verified`, `picture`, `locale`, `updated_at`.

Claims de sesión y contexto

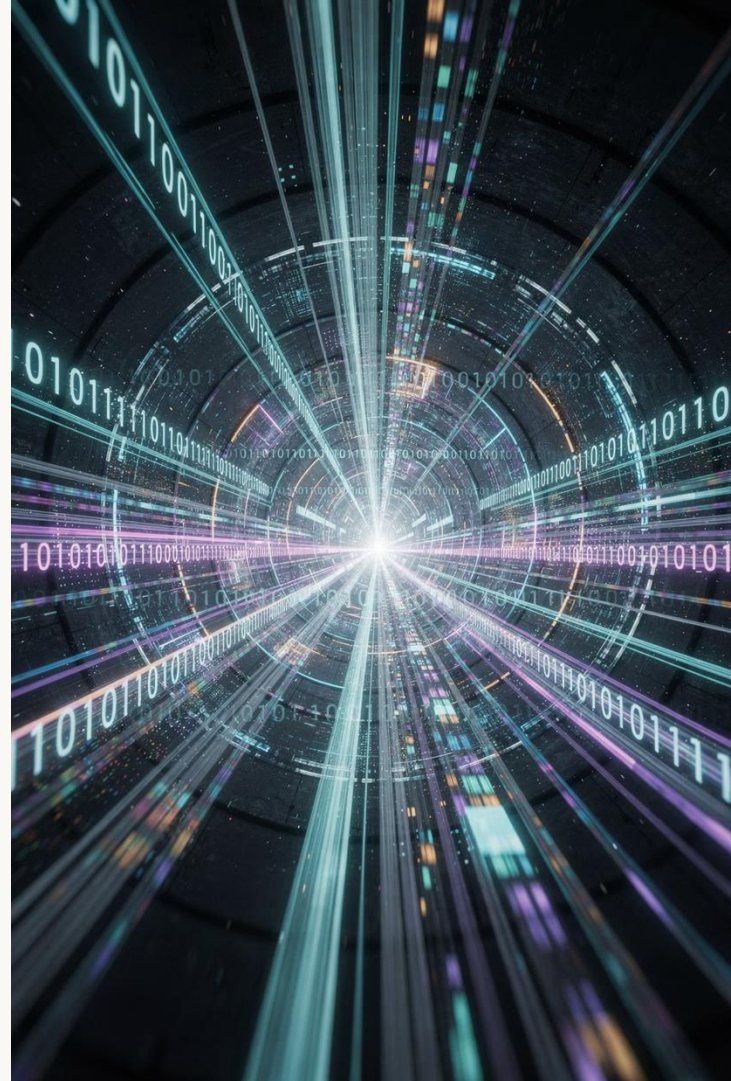
`iss` (issuer), `aud` (audience), `exp` (expiration), `iat` (issued at), `auth_time`, `nonce` (protección contra replay attacks), `acr` (Authentication Context Class Reference).

Claims personalizados

Los IdPs permiten añadir claims propios (roles, departamento, tenant_id...) mediante mappers o transformaciones. Por convención se usa un namespace URI como prefijo para evitar colisiones.

JSON Web Token (JWT)

El formato estándar
para tokens de
seguridad



¿Qué es un JWT?

Un **JSON Web Token** (RFC 7519) es un formato compacto y autocontenido para transmitir información de forma segura entre partes como un objeto JSON. Los JWT pueden ser **firmados** (JWS) o **cifrados** (JWE).

Header

Metadatos del token: tipo (JWT) y algoritmo de firma (RS256, HS256, ES256).

```
{"alg": "RS256", "typ": "JWT" }
```

Payload

Claims: datos del usuario y del token (sub, iss, exp, iat, roles, scopes...). Este contenido es legible en Base64URL.

```
{"sub": "123", "roles": ["admin"] }
```

Signature

Firma criptográfica calculada sobre Header + Payload. Garantiza la integridad del token. Solo el emisor puede generarla correctamente.

Firma y verificación de JWT

La elección del algoritmo de firma tiene implicaciones de seguridad críticas. Existen dos familias principales:

Algoritmos asimétricos (recomendados)

- **RS256:** RSA + SHA-256. El IdP firma con clave privada, los Resource Servers verifican con clave pública (JWKS).
- **ES256:** ECDSA + SHA-256. Más eficiente que RSA, misma seguridad con claves más cortas.
- **PS256:** RSASSA-PSS. Más robusto que RS256 frente a ciertos ataques.

Algoritmos simétricos (usar con precaución)

- **HS256:** HMAC + SHA-256. Usa un secreto compartido entre emisor y verificador. No escalable en microservicios: todos los servicios necesitan el secreto.



Nunca uses `alg: none`. Algunos parsers legacy aceptan tokens sin firma si se especifica este valor — vulnerabilidad crítica.

Validación correcta de JWT

Recibir un JWT válido no es suficiente: hay que **validar todos sus claims**. Un token bien firmado pero con claims incorrectos puede ser reutilizado o ser válido para otro servicio.

01

Verificar la firma

Obtener la clave pública del JWKS endpoint del IdP y verificar la firma criptográfica del token.

02

Validar iss

El issuer debe coincidir exactamente con el IdP esperado. Rechazar tokens de emisores desconocidos.

03

Validar exp e iat

Comprobar que el token no ha expirado y que su fecha de emisión es razonable (protección contra tokens futuros).

04

Validar aud

El audience debe incluir el identificador de tu servicio. Evita que tokens emitidos para otro servicio sean aceptados en el tuyo.

Ciclo de vida del JWT: expiración y refresco



❗ El **refresh token rotation** es una práctica recomendada: cada vez que se usa un refresh token, se invalida y se emite uno nuevo. Si el refresh token original se usa de nuevo, el servidor detecta una reutilización sospechosa y puede revocar toda la sesión.

JWT: ventajas y limitaciones

Ventajas

- **Stateless:** el Resource Server no necesita consultar una base de datos para validar el token
- Escalable horizontalmente sin estado compartido
- Autocontenido: los claims necesarios viajan en el propio token
- Estándar ampliamente soportado en todas las plataformas

Limitaciones y riesgos

- **Irrevocabilidad:** un JWT firmado es válido hasta su expiración, aunque el usuario cierre sesión
- Payload visible en Base64 (no cifrado por defecto) — no incluyas datos sensibles
- Tokens muy grandes pueden impactar el rendimiento de red
- Requiere estrategia de revocación (token blacklist, expiración corta + refresh rotation)

Estrategias de revocación de tokens

1

Expiración corta

Access tokens con TTL de 5–15 minutos. El daño de un token comprometido queda limitado en el tiempo. La estrategia más sencilla y efectiva.

2

Token Blacklist

Almacén centralizado (Redis) de JTIs (`jti claim`) revocados. El Resource Server consulta la blacklist en cada petición. Introduce estado, pero permite revocación inmediata.

3

Introspection Endpoint

El Resource Server consulta al Authorization Server si el token sigue siendo válido (RFC 7662). Añade latencia pero garantiza revocación en tiempo real.

4

Refresh Token Rotation

Cada uso del refresh token invalida el anterior y emite uno nuevo. Detecta reutilización maliciosa y permite revocar la sesión completa.

Relación entre OAuth 2.0, OIDC y JWT

Es un error común confundir estos tres componentes. Tienen roles distintos y complementarios en una arquitectura de seguridad moderna.

Aspecto	OAuth 2.0	OpenID Connect	JWT
¿Qué es?	Framework de autorización delegada	Protocolo de autenticación	Formato de token
Propósito	Autorizar acceso a recursos	Verificar identidad del usuario	Transportar claims
Basado en	RFC 6749/6750	Sobre OAuth 2.0	RFC 7519
Token principal	Access Token	ID Token	Ambos pueden ser JWT
¿Autenticación?	No directamente	Sí	No (es solo formato)

Taller Práctico

Análisis de arquitectura de autenticación corporativa

Aplicamos los conceptos de la sesión a un escenario real de empresa



Contexto del taller

Trabajamos sobre una empresa ficticia, **«FinSecure»**, una fintech que opera en España y está modernizando su infraestructura de identidad. Actualmente sufren varios problemas de seguridad que debéis identificar y resolver.

Situación actual

Cada microservicio gestiona sus propias credenciales. Los usuarios tienen una cuenta distinta por aplicación. No existe SSO. Los tokens no tienen expiración configurada.

Objetivo del taller

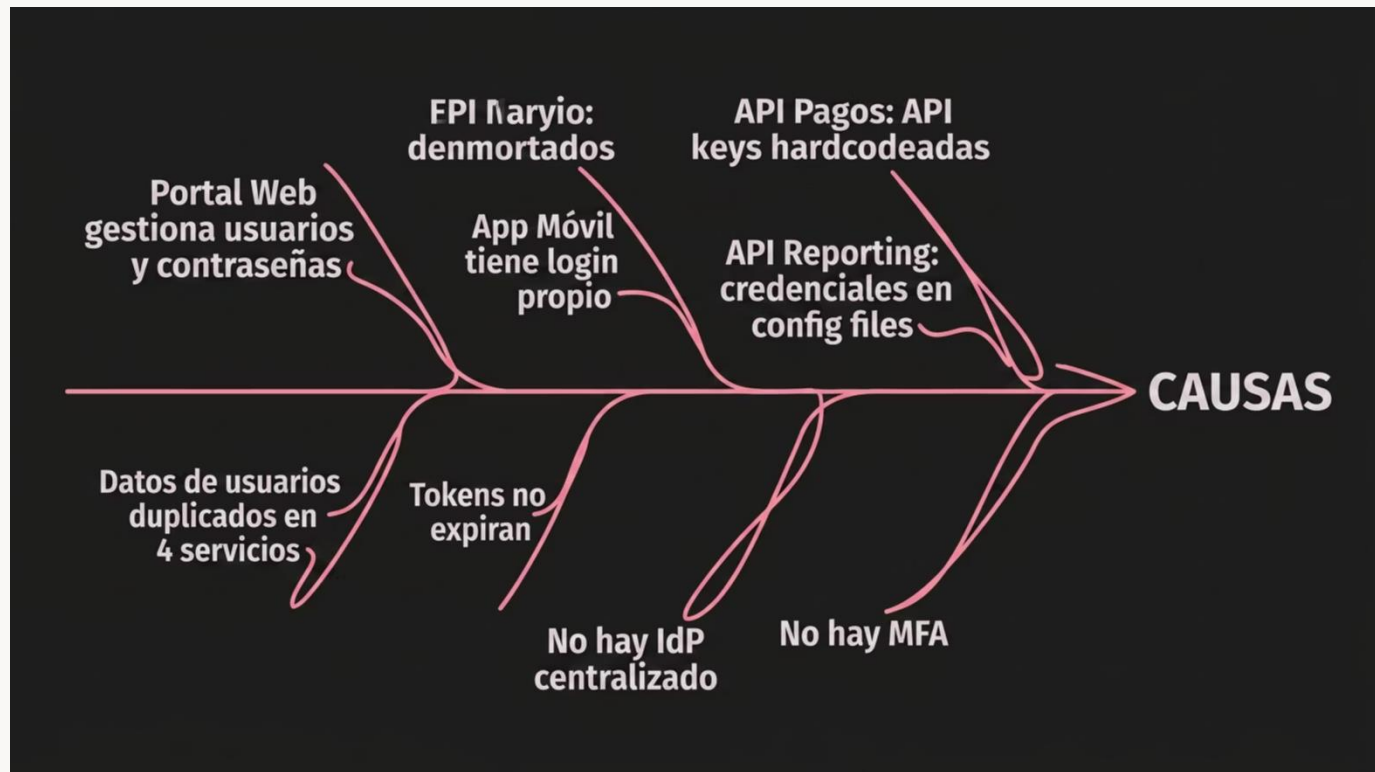
Diseñar una arquitectura de identidad centralizada con SSO, definir los flujos OAuth2/OIDC adecuados para cada tipo de cliente e identificar los fallos en el sistema actual.

Entregable

Diagrama de arquitectura anotado + listado de decisiones de diseño + justificación de los flujos OAuth2 elegidos para cada componente.

Arquitectura actual de FinSecure

Antes de proponer mejoras, debéis entender el sistema actual y sus problemas. Identificad los **puntos de fallo** en el siguiente esquema.



Análisis de vulnerabilidades — FinSecure actual

→ Credenciales duplicadas y fragmentadas

Cuatro almacenes de usuarios distintos implican cuatro superficies de ataque, cuatro políticas de contraseña diferentes y cuatro lugares donde gestionar bajas de empleados (riesgo de cuentas huérfanas).

→ Tokens sin expiración

Un token comprometido es válido indefinidamente. No existe forma de revocar el acceso una vez que un atacante obtiene un token.

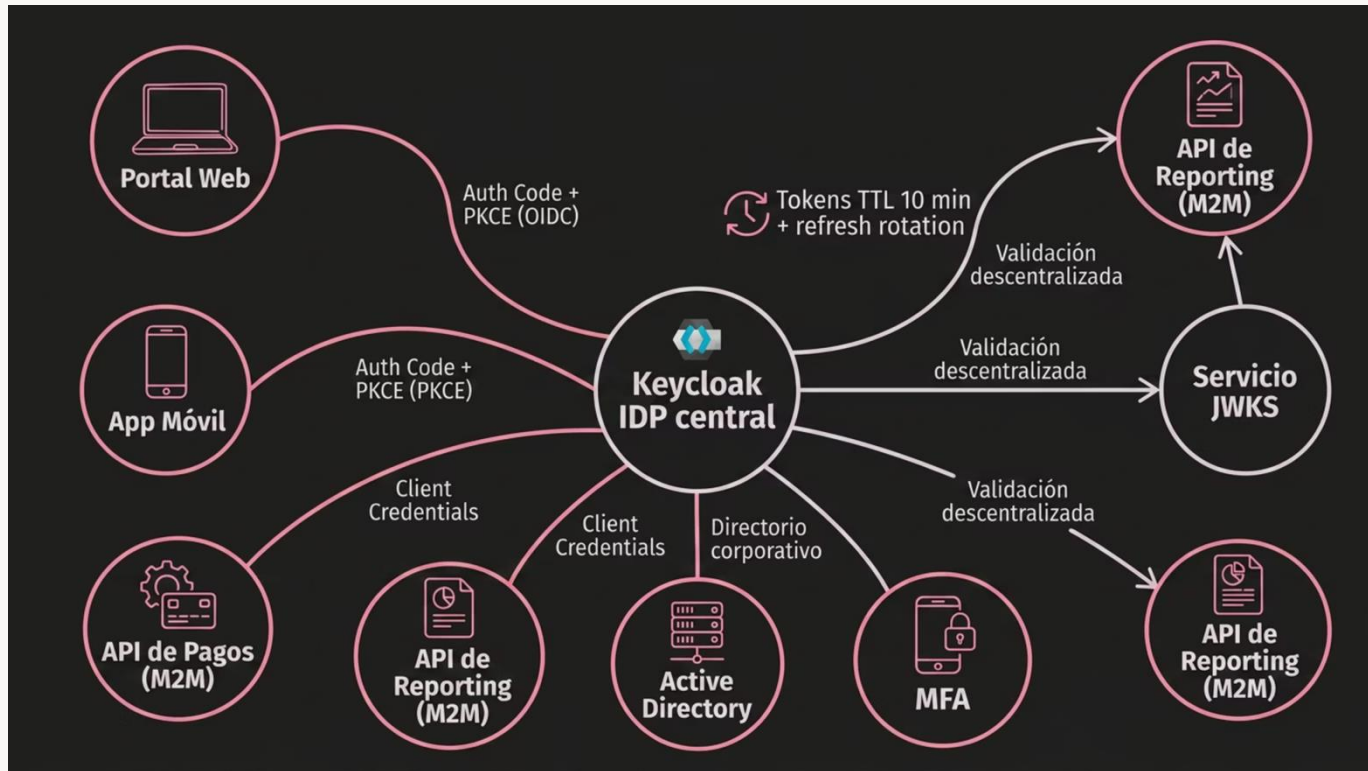
→ API keys hardcodeadas

Las API keys en código fuente o ficheros de configuración son un vector de ataque clásico. Cualquier persona con acceso al repositorio puede obtenerlas. No son rotables fácilmente.

→ Ausencia de MFA

Sin segundo factor, el compromiso de una contraseña es suficiente para acceder a los sistemas financieros. Incumple además los requisitos de PSD2 para autenticación reforzada (SCA).

Arquitectura propuesta — FinSecure v2



Decisiones de diseño — FinSecure v2

1

Keycloak como IdP central

Open source, on-premise (requisito regulatorio fintech), compatible con OIDC/SAML, soporta federación con Active Directory y MFA nativo.

2

Authorization Code + PKCE para clientes de usuario

Portal Web y App Móvil son clientes públicos. PKCE es obligatorio para proteger el authorization code en tránsito.

3

Client Credentials para M2M

Las APIs internas que se comunican entre sí sin intervención de usuario usan Client Credentials con secretos gestionados en Vault (no en código).

4

Tokens de corta vida + refresh rotation

Access tokens de 10 minutos. Refresh tokens con rotación activa. La revocación de sesión es inmediata al invalidar el refresh token.

Identificación de componentes de seguridad

En una aplicación distribuida moderna, los componentes de seguridad están presentes en múltiples capas. Saber identificarlos es clave para auditarlos y mantenerlos correctamente.



API Gateway

Primera línea de defensa. Valida tokens, aplica rate limiting, gestiona CORS y enruta al servicio correcto. Ej.: Kong, AWS API Gateway, Azure APIM.



Identity Provider (IdP)

Gestiona el ciclo de vida de identidades y sesiones, emite tokens y expone los endpoints OIDC estándar (authorization, token, userinfo, JWKS, discovery).



Resource Server

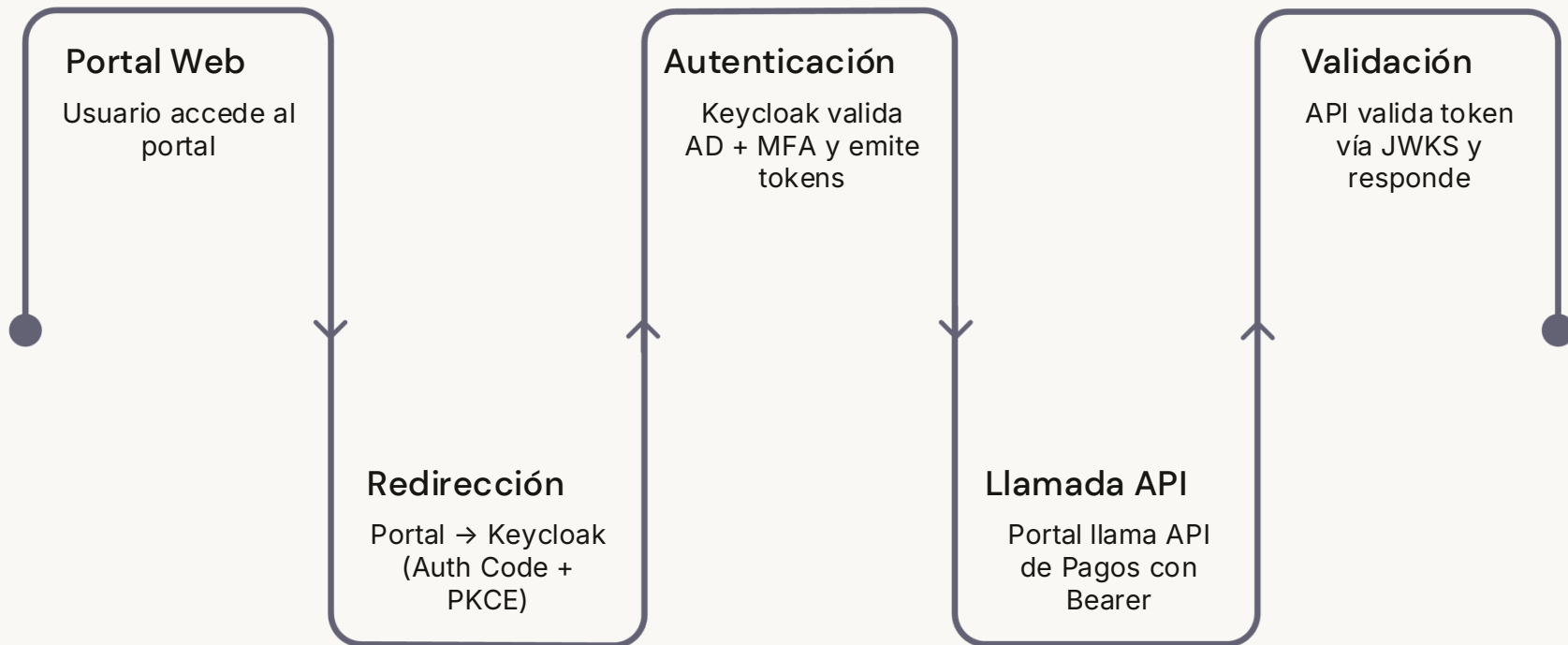
Cada microservicio que expone recursos protegidos. Valida el JWT localmente usando la clave pública del IdP (JWKS) sin necesidad de llamar al IdP en cada petición.



Secrets Manager

HashiCorp Vault, AWS Secrets Manager o Azure Key Vault. Gestiona las credenciales de los clientes OAuth2, certificados TLS y claves de firma de forma segura y auditable.

Flujo completo en la arquitectura FinSecure v2



Este flujo ilustra cómo los cuatro roles de OAuth2 (Resource Owner, Client, Authorization Server y Resource Server) participan en una transacción real. La validación del token en el paso 5 es completamente local: no requiere llamada adicional al IdP.

Errores frecuentes en implementaciones reales

Almacenar tokens en localStorage

Vulnerable a ataques XSS. Los access tokens deben almacenarse en memoria; los refresh tokens en cookies `HttpOnly; Secure; SameSite=Strict`.

No validar el claim aud

Un atacante puede usar un token emitido para otro servicio si el Resource Server no valida el audience. El resultado es un «confused deputy attack».

Usar el ID Token como Access Token

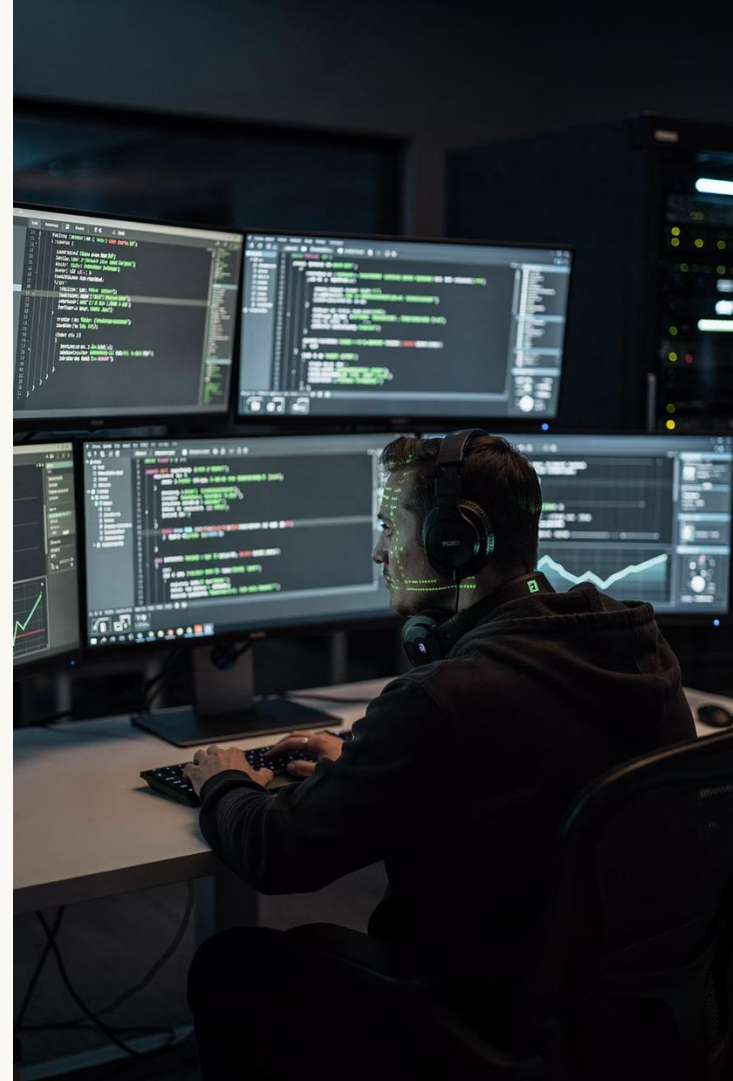
El ID Token es para el cliente, no para las APIs. Enviarlos al Resource Server viola la especificación OIDC y expone datos de identidad innecesariamente.

Secretos en variables de entorno sin rotación

Las variables de entorno son accesibles a cualquier proceso del mismo host y no se rotan automáticamente. Usar un Secrets Manager dedicado.

Buenas prácticas de implementación

Principios que todo arquitecto/a de seguridad debe interiorizar antes de escribir la primera línea de código de autenticación.



Principios de diseño seguro para identidad

No implementes tu propio sistema de autenticación

Usa bibliotecas y frameworks auditados (Spring Security, ASP.NET Identity, Passport.js). La criptografía mal implementada es peor que no tenerla.

Defense in depth

La autenticación no es la única línea de defensa. Combínala con autorización granular, auditoría, cifrado en tránsito y en reposo, y monitorización de anomalías.

Principio de mínimo privilegio

Los tokens deben contener únicamente los scopes y claims necesarios para la operación solicitada. Nunca emitas tokens con más permisos de los requeridos.

Fallar de forma segura

En caso de error o duda, denegar el acceso. Los mensajes de error no deben revelar información sobre la estructura interna del sistema.

Configuración de Keycloak para el taller

Pasos esenciales para configurar un realm de Keycloak equivalente al escenario FinSecure v2. Estos son los mismos pasos que realizaréis en el laboratorio.

01

Crear el Realm

Un realm es un espacio de identidad aislado. Crea `finsecure-realm` con la configuración de sesión: access token TTL 10 min, refresh token TTL 1 hora, refresh token rotation activado.

03

Definir roles y mappers

Crear roles de realm (`admin`, `analyst`, `readonly`) y configurar mappers para incluir los roles como claims en el access token.

02

Configurar clientes OAuth2

Client `portal-web` (public, Authorization Code + PKCE), client `api-pagos` (confidential, Client Credentials). Definir redirect URIs y scopes permitidos.

04

Activar MFA

Configurar la política de autenticación del realm para requerir OTP (TOTP/HOTP) en el primer login y en accesos desde IPs no reconocidas.

Validación de JWT en Spring Boot

Configuración mínima para que un Resource Server Spring Boot valide tokens JWT emitidos por Keycloak usando el JWKS endpoint.

```
# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://keycloak.finsecure.io/realms/finsecure-realm
          # Spring descarga automáticamente el JWKS del discovery endpoint
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/payments/**").hasRole("analyst")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtConverter())))
            );
        return http.build();
    }
}
```

Validación de JWT en ASP.NET Core

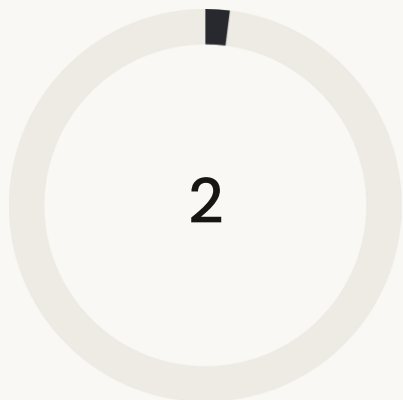
Configuración equivalente en el ecosistema .NET para validar tokens OIDC emitidos por Azure Entra ID o Keycloak.

```
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "https://keycloak.finsecure.io/realms/finsecure-realm";
        options.Audience = "api-pagos";
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer          = true,
            ValidateAudience        = true,
            ValidateLifetime         = true,
            ValidateIssuerSigningKey = true,
            ClockSkew                = TimeSpan.FromSeconds(30)
        };
    });

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AnalystOnly", policy =>
        policy.RequireClaim("roles", "analyst"));
});
```

 El SDK descarga automáticamente las claves públicas desde el JWKS endpoint del Authority y las rota en caché según la cabecera `Cache-Control` del servidor.

Puntos de control: ¿qué hemos aprendido?



Conceptos base

Autenticación (AuthN) y Autorización (AuthZ):
diferentes en propósito, momento y
mecanismo.



Estándares clave

OAuth 2.0, OpenID Connect y JWT: roles
complementarios en el ecosistema de
identidad moderno.



Flujos OAuth2

Authorization Code + PKCE, Client
Credentials, Device Code — cada uno para
su caso de uso.

Preguntas de reflexión para el taller

Utiliza estas preguntas como guía para el análisis y la discusión en grupo:

Pregunta 1

¿Por qué el flujo Authorization Code sin PKCE es inseguro para aplicaciones de página única (SPA)? ¿Qué ataque específico mitiga PKCE?

Pregunta 2

En la arquitectura FinSecure, la API de Reporting necesita acceder a datos de usuario. ¿Debería usar Client Credentials o Authorization Code? Justifica tu respuesta.

Pregunta 3

Un desarrollador propone almacenar el `jwt` de todos los tokens emitidos en Redis para poder revocarlos. ¿Qué ventajas e inconvenientes tiene este enfoque frente a usar tokens de muy corta vida?

Pregunta 4

¿Qué información NO debería incluirse nunca en el payload de un JWT? ¿Por qué? ¿En qué casos usarías JWE (cifrado) en lugar de JWS (firmado)?

Recursos de referencia

Especificaciones oficiales (IETF/OpenID)

- **RFC 6749** — The OAuth 2.0 Authorization Framework
- **RFC 6750** — Bearer Token Usage
- **RFC 7519** — JSON Web Token (JWT)
- **RFC 7636** — PKCE for OAuth Public Clients
- **RFC 7662** — OAuth 2.0 Token Introspection
- **RFC 9068** — JWT Profile for OAuth 2.0 Access Tokens
- **OpenID Connect Core 1.0** — openid.net/specs

Guías y herramientas prácticas

- **OAuth 2.0 Security BCP** (RFC 9700) — mejores prácticas actualizadas
- **jwt.io** — decodificador y debugger de JWT online
- **OWASP ASVS** — Application Security Verification Standard
- **Keycloak Docs** — keycloak.org/documentation
- **Spring Security OAuth2** — docs.spring.io
- **Microsoft Identity Platform** — learn.microsoft.com

Glosario de términos clave

Término	Definición
AuthN	Autenticación: verificación de identidad (¿quién eres?)
AuthZ	Autorización: control de acceso a recursos (¿qué puedes hacer?)
IdP	Identity Provider: entidad que gestiona identidades y emite tokens
SSO	Single Sign-On: autenticación única para múltiples aplicaciones
PKCE	Proof Key for Code Exchange: extensión de seguridad para OAuth2 en clientes públicos
JWKS	JSON Web Key Set: endpoint con claves públicas para verificar tokens JWT
Claim	Afirmación sobre una entidad incluida en un token (sub, roles, email...)
Scope	Permiso solicitado por el cliente para acceder a recursos específicos
Introspection	Proceso de verificar la validez de un token opaco consultando al Authorization Server
mTLS	Mutual TLS: autenticación bidireccional mediante certificados X.509



Conceptos avanzados a explorar

Temas que amplían los fundamentos de esta sesión y serán relevantes en sesiones posteriores del curso.

Token Binding y Sender-Constrained Tokens

Una limitación fundamental de los Bearer tokens es que cualquier entidad que los posea puede usarlos. Las técnicas de **token binding** vinculan el token a una clave criptográfica del cliente, eliminando el riesgo de reutilización por un atacante que intercepte el token.

DPoP (Demonstrating Proof of Possession)

RFC 9449. El cliente firma cada petición HTTP con una clave privada efímera. El servidor verifica que la firma corresponde a la clave pública declarada en el token. Adoptado en FAPI 2.0.

mTLS Certificate-Bound Tokens

RFC 8705. El access token incluye el thumbprint del certificado TLS del cliente. El Resource Server verifica que la conexión TLS usa ese certificado exacto.

¿Cuándo usarlos?

Obligatorios en FAPI (Financial-grade API), recomendados en APIs que manejan datos financieros o sanitarios. Cada vez más exigidos por regulaciones como PSD2.

Federación de identidades

En entornos empresariales es frecuente que los usuarios ya tengan identidades en un directorio corporativo (Active Directory, LDAP). La **federación de identidades** permite que el IdP delegue la autenticación a ese directorio sin duplicar las credenciales.

1

Identity Federation

El IdP (Keycloak/Azure AD) se conecta al directorio corporativo mediante LDAP, AD o un proveedor SAML externo.

2

User Federation

Los usuarios del directorio son visibles en el IdP sin necesidad de importarlos. El IdP delega la validación de contraseñas al directorio.

3

Token Translation

El IdP emite tokens OIDC/OAuth2 estándar independientemente del protocolo usado internamente con el directorio (LDAP, Kerberos...).

Zero Trust y el papel de la identidad

Del modelo perimetral al Zero Trust

El modelo de seguridad perimetral («confío en todo lo que está dentro de mi red») ha quedado obsoleto con la adopción de cloud, microservicios y trabajo remoto.

En un modelo **Zero Trust**, la identidad se convierte en el nuevo perímetro: *«nunca confíes, siempre verifica»*. Cada petición —interna o externa— debe autenticarse y autorizarse explícitamente.

- Verificación continua de identidad y contexto
- Mínimo privilegio en cada token y sesión
- Microsegmentación de acceso entre servicios
- mTLS para comunicaciones servicio-a-servicio

Implicaciones para OAuth2/OIDC

- Access tokens de muy corta vida (≤ 5 min)
- Evaluación de contexto en cada petición (IP, device, hora)
- Step-up authentication para operaciones sensibles
- Token binding obligatorio (DPoP/mTLS)
- Auditoría continua y detección de anomalías

FAPI — Financial-grade API Security Profile

Estándar de la OpenID Foundation que eleva los requisitos de seguridad de OAuth2/OIDC para APIs financieras. Relevante para proyectos FinTech como FinSecure.

FAPI 1.0 (Baseline + Advanced)

Requisitos adicionales sobre OAuth2: PAR (Pushed Authorization Requests), JAR (JWT-Secured Authorization Requests), mTLS o DPoP obligatorio, sin flujo Implicit.

FAPI 2.0

Simplifica FAPI 1.0 eliminando opciones confusas. Adopta Authorization Code + PKCE como flujo único, DPoP como mecanismo de token binding preferido.

Open Banking en España (PSD2)

PSD2 requiere autenticación reforzada (SCA) para pagos. Los esquemas de Open Banking (Berlin Group NextGenPSD2) se basan en FAPI para las APIs de TPP (Third Party Providers).

Checklist de seguridad — Sesión 1

Antes de dar por completada la implementación de autenticación en un sistema, verificad estos puntos:

- | | |
|---|--|
| ☒ AuthN y AuthZ implementados como componentes independientes | ☐ Access tokens con TTL ≤ 15 minutos |
| ☒ IdP centralizado (no autenticación ad-hoc por servicio) | ☐ Refresh token rotation activado |
| ☒ SSO configurado con SLO (Single Logout) funcional | ☐ Validación de <code>iss</code> , <code>aud</code> , <code>exp</code> , firma en cada servicio |
| ☒ Flujo OAuth2 correcto para cada tipo de cliente | ☐ MFA habilitado para usuarios privilegiados |
| ☒ PKCE activado en todos los clientes públicos | ☐ Tokens almacenados de forma segura (no en localStorage) |
| ☒ Secretos de clientes en Secrets Manager (no en código) | ☐ Auditoría de eventos de autenticación activada |

Conexión con las sesiones siguientes

Los conceptos de esta sesión son la base sobre la que construiremos en el resto del curso.

Sesión 1

Fundamentos: AuthN, AuthZ, SSO, OAuth2, OIDC, JWT.
Arquitectura de identidad centralizada.

Sesión 3 →

Seguridad en APIs: rate limiting, CORS, SSL/TLS, headers de seguridad, WAF y gestión de secretos con Vault.

Sesión 2 →

Spring Security y ASP.NET Identity: implementación práctica de los flujos OAuth2. Configuración avanzada de Resource Servers. Gestión de tokens en código.

Sesión 4 →

Vulnerabilidades comunes (OWASP Top 10), SQL Injection, XSS, CSRF, XXE y simulación de ataques.

Resumen ejecutivo — Sesión 1



Identidad centralizada

Un IdP único elimina credenciales fragmentadas, unifica políticas y facilita la auditoría. SSO mejora UX y seguridad simultáneamente.



OAuth 2.0 bien aplicado

Cada tipo de cliente tiene su flujo: Authorization Code + PKCE para usuarios, Client Credentials para M2M. Los flujos deprecados no deben usarse.



JWT: potente pero cuidadoso

Validar siempre firma, iss, aud y exp. Expiración corta + refresh rotation. Nunca datos sensibles en el payload sin cifrado.



Defensa en profundidad

La autenticación es solo la primera capa. Combinarla con autorización granular, auditoría, secretos gestionados y monitorización continua.



Próximos pasos

Sesión 2: Spring Security & ASP.NET Identity

Implementación práctica de los flujos OAuth2 estudiados hoy. Configuración avanzada de Resource Servers, gestión de roles con tokens y taller de integración con Keycloak en Java y .NET.

SESIÓN 2

SPRING SECURITY

ASP.NET IDENTITY